Phonebook Directory

Project contains 6 main modules:

1. Add Contact (Sorted Insertion)

2. Search Contact (Unified Search by Name/Phone)

3. Update Contact

4. Delete Contact

5. Display Contacts

6. Statistics & File Handling (Save/ Load Binary Data)

Requirements:

Hardware Requirements:

Computer/Laptop
Minimum 4GB RAM

Software Requirements:

C++ Compiler (e.g., Turbo C++, Code::Blocks, VS Code)
Operating System (Windows/Linux/Mac)

Methodology:

The system is implemented using a Doubly Linked List, where:
Each node stores:

Name
Phone number
Pointer to next node
Pointer to previous node

Working Approach:

Contacts are inserted in sorted order

Search, update, and delete operations traverse the list

File handling ensures data persistence

Destructor automatically saves data before exit.

Algorithm:

Phonebook Directory

1. Data Structure Organization
 1. Define Contact Record: Create a structure ContactData to store a fixed-length character array for Name (50 bytes) and Phone (15 bytes).
 2. Define Node:** Create a doubly linked list `Node` containing the `ContactData`, a pointer to the `Next` node, and a pointer to the `Previous` node.
 3. Initialize:** Set the global/class `Head` pointer to `NULL`.

2. Sorted Insertion Algorithm (Add Contact)

Step 1: Start.
Step 2: Create a `NewNode` and assign the user-provided `Name` and `Phone`.
Step 3: If the list is empty **OR** the `NewNode->Name` is alphabetically smaller than `Head->Name`:
    * Link `NewNode->Next` to `Head`.
    * If `Head` is not NULL, set `Head->Prev` to `NewNode`.
    * Update `Head` to `NewNode`.
Step 4: Else, traverse the list using a temporary pointer `Curr` until:
    `Curr->Next` is NULL **OR** `Curr->Next->Name` is alphabetically greater than `NewNode->Name`.
Step 5: Perform the insertion:
    Set `NewNode->Next` to `Curr->Next`.
    Set `NewNode->Prev` to `Curr`.
    If `Curr->Next` is not NULL, set `Curr->Next->Prev` to `NewNode`.
    Set `Curr->Next` to `NewNode`.
Step 6: Stop.

3. Unified Search and Delete Algorithm

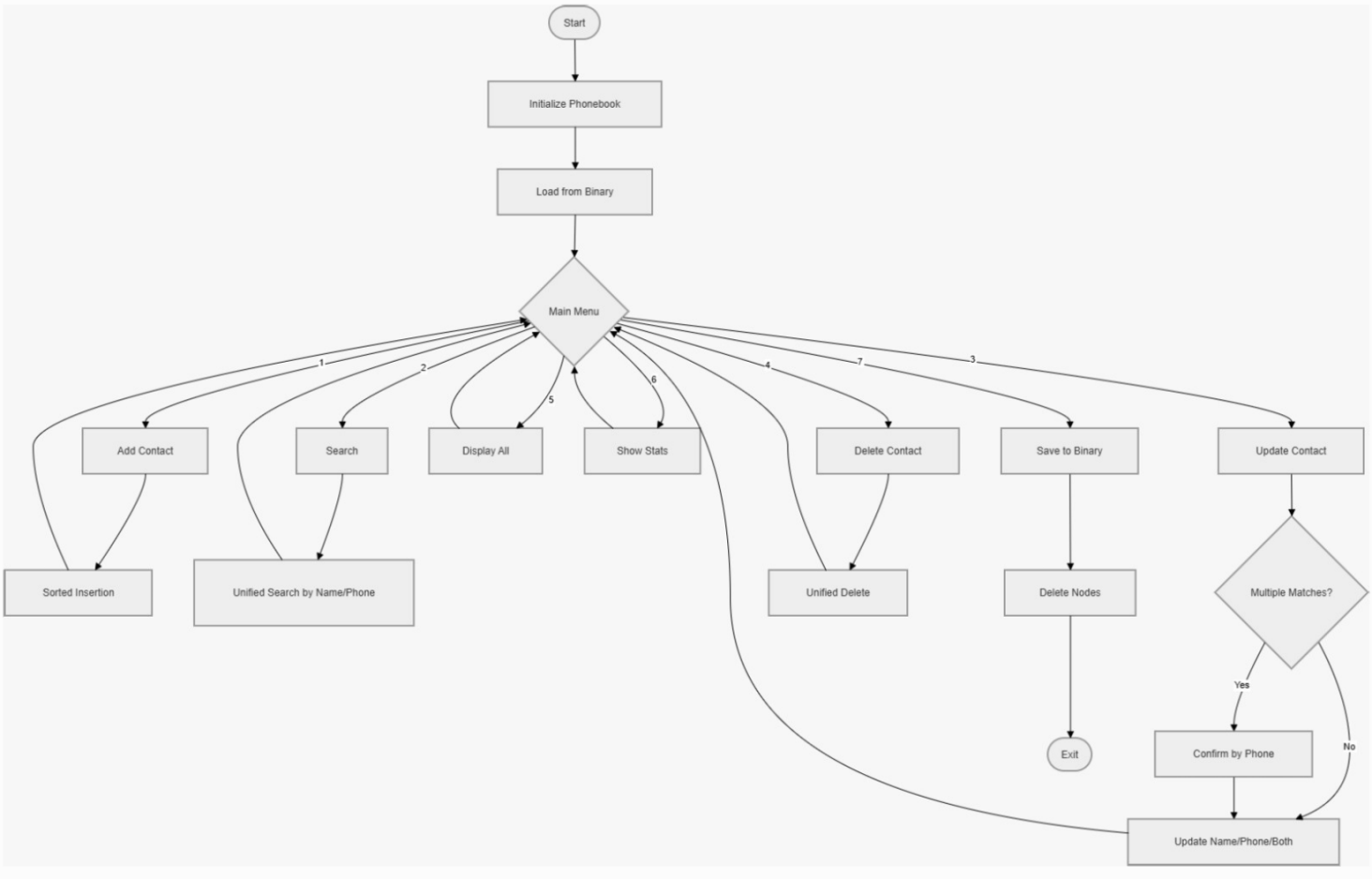
Step 1: Start.
Step 2: Input the `Query` (string).
Step 3: Traverse the list from `Head` to the end.
Step 4:** For each node, compare `Query` with `Node->Name` and `Node->Phone`.
Step 5 (If Search):** If a match is found, display the contact details.
Step 6 (If Delete):** If a match is found:
    Bypass the current node by linking `Node->Prev->Next` to `Node->Next`.
    Link `Node->Next->Prev` to `Node->Prev`.
    If the node is `Head`, update `Head` to `Node->Next`.
    Free/Delete the memory of the current node.
Step 7: Stop.

4. Integrated Update Algorithm

Step 1: Search for the record using the provided `Query`.
Step 2: If multiple matches exist, prompt for the exact `Phone` number to isolate the record.
Step 3: If updating the Number: Simply overwrite the `Phone` array in the current node.
Step 4: If updating the Name:
    Store the new Name and existing Phone.
    Delete the old node (to maintain alphabetical integrity).
    Re-insert the contact using the Sorted Insertion Algorithm.
Step 5:Stop.

5. File Persistence Algorithm (Binary I/O)
Save Operation:
    1. Open `phonebook.dat` in binary output mode.
    2. Traverse the linked list from `Head`.
    3. Write the `ContactData` block of each node sequentially to the file.
Load Operation:
    1. Open `phonebook.dat` in binary input mode.
    2. While there is data to read, pull one `ContactData` block into a temporary variable.
    3. Pass the data to the **Sorted Insertion Algorithm** to reconstruct the list in memory.

Flowchart:


Start
Initialize Phonebook
Load from Binary
Main Menu
1
2
5
6
4
7
3
Add Contact
Search
Display All
Show Stats
Delete Contact
Save to Binary
Update Contact
Sorted Insertion
Unified Search by Name/Phone
Unified Delete
Delete Nodes
Multiple Matches?
Yes
No
Exit
Confirm by Phone
Update Name/Phone/Both